

M H V A N



AI Context Exporter

Official Documentation

AI Context Exporter For AI (AI Context Exporter) - Official Documentation V1.1 (CURRENT)

Overview & Business Value

Asset Context Exporter For AI (AI Context Exporter) is a comprehensive C++ solution designed for solo developers, indie creators, and AAA studios looking to standardize their Data-Driven Pipeline. The tool translates Unreal Engine binary files (.uasset, .umap) into highly optimized, structured text formats (JSONL). It serves as the ultimate bridge between project architecture and Large Language Models (LLMs) or Agentic AI, empowering both non-programmers and

seasoned engineers to build, audit, and scale their projects efficiently.

Secure & Air-Gapped Operation: The tool operates 100% offline within the studio's intranet. There are no external dependencies, third-party APIs, no HTTP request or hidden telemetry. Proprietary code and project logic never leave the local hardware, ensuring strict compliance with corporate Non-Disclosure Agreements (NDAs).

WHAT IS NEW IN V1.1

~ **EXPORT MODES** (replaces the old Monolithic / File Splitting checkboxes)

A single, unambiguous 4-way selector defines the physical layout of the export:

- **Standard:** One .jsonl file per asset. Ideal for browsing and attaching specific assets.

- **Fragmented:** One file per asset, split into parts capped at "Max File Size (MB)".

Prevents context-window overflows on large payloads.

- **Consolidated:** One Master_<Module>.jsonl per module gathering every asset of that module in a single self-describing JSONL file (one line per asset). An AI can read the entire module context without walking hundreds of files. Individual files are NOT exported in this mode.

- **Monolithic:** A single project-wide Master_Project_Context.jsonl for RAG pipelines, vector databases and autonomous agents.

"Preserve Content Drawer Hierarchy" only applies to Standard and Fragmented modes.

~ **ANIMATION SYSTEM MODULE** (complete redesign of the former "Animations" module)

The module no longer relies on raw coordinate dumps (bulk keyframe/spatial arrays are now always omitted). Instead it exports the structural data an AI can actually reason about:

- **Skeletons:** full bone tree (name, parent, compact local position), sockets, virtual bones, slot groups, animation curves, and compatible-skeleton lists. Per-bone translation retargeting is written for every bone that deviates from the default mode, so a retargeting mismatch can be traced bone by bone.

- **Skeletal Meshes:** mesh-side bone hierarchy (cross-checkable against the Skeleton), skeleton/physics-asset bindings, LOD count, sockets, morph target names and material slots.
- **Animation Blueprints:** TargetSkeleton binding, class Details (CDO delta), and every graph including nested State Machines (states, conduits, entries, transitions with From/To, blend Details and transition rule ASTs), AnimGraphs with per-node Details (play rates, blend times, linked sequences - only user-modified values), state graphs and event graphs. Miswired transitions become visible in plain text.
- **Blend Spaces (1D/2D/Aim Offsets):** axes (name, min, max, grid divisions), every sample (animation + position) and interpolation settings.
- **IK Rigs & IK Retargeters:** exported via reflection (goals, solvers, chain settings) with no hard plugin dependency.
- **Control Rig:** RigVM graph AST (nodes, connections, values) so an AI can read the procedural rig logic that drives the character.
- **Physics Assets (ragdolls):** compact per-body data (bone, physics type, collision shape summary) and per-constraint data (bone pair, linear/angular motion + limits, which drives are enabled). A Physics Asset is authored per skeleton and is part of the character rig, so it now lives in the Animations module, next to the Skeleton and Skeletal Mesh. It is exported as a purpose-built compact record.
- **Anim Sequences/Montages:** skeleton binding, length, sampled key count, notifies, sections, slots and curve metadata (structure, not raw keys).

~ DATA MODULE ADDITIONS (V1.1)

- **User Defined Structs:** every field with its name and type, so an AI knows the exact shape of your custom data types instead of guessing them.
- **User Defined Enums:** every entry with value and display name, so an AI uses your real enum options rather than inventing them.
- **Gameplay Tag Dictionary:** the full project tag hierarchy is exported once as a single _GameplayTags file, so an AI uses your real tags instead of inventing them.

~ FURTHER ADDITIONS (V1.1)

- Physics Assets are now exported with the full tuning an AI needs to diagnose a glitchy ragdoll: per-body collision shape dimensions and damping/mass overrides, and per-constraint angular/linear DRIVE parameters (stiffness, damping, max force, drive mode), soft-limit values, projection tolerances and parent-dominance. Still compact (a few KB), not a raw reflection dump.
- Static Mesh metadata (World Levels module): LOD count, bounds, LOD0 triangle count, material slots, sockets and collision complexity - metadata only, never geometry.
- PCG Graphs (Blueprints module): node list and settings for procedural generation graphs, detected without a hard PCG plugin dependency.
- GAS: Gameplay Ability / Gameplay Effect / Attribute Set blueprints are detected and tagged; their full data (tags, cooldowns, modifiers) already exports via the CDO in the Blueprints module, with no hard GameplayAbilities dependency.

~ UI / FOLDER COHERENCE

Exported folder numbering now mirrors the UI module numbering 1:1:

01_Source_Code,
02_Blueprints,
03_Data_Assets,
04_World_Levels,
05_VFX_Materials,
06_Animations,
07_AI_Logic,
08_Audio,
09_Input_Mappings,
10_Physics

(plus 00_Project_Manifest for the path index, folder tree AND the project dependency graph, which is a project-wide architecture artifact rather than a single blueprint).

DataTables/StringTables/Structs/Enums/GameplayTags live inside 03_Data_Assets.

Skeletons, Skeletal Meshes, Physics Assets and Control Rigs inside 06_Animations.

10_Physics holds Physical Materials; the source snapshot and .ini configuration share

01_Source_Code. Output folders are always generated in English.

Inside the directory you choose, exports are organised as

ASSET CONTEXT EXPORTER FOR AI_Exports / <Project> / <Date> / <Time>_Export.

~ WORLD LEVELS NOW RESPECT THE EXPORT MODE

World Levels used to always batch actors into consolidated multi-actor files (a legacy workaround from before export modes existed, meant for worlds with hundreds of thousands of components). Now each level respects the chosen export mode: Standard and Fragmented write one file per actor (plus a per-level "<Level>__WorldInfo" file with World Settings and World Partition data), while Consolidated and Monolithic gather every actor into the single module/project file. World Levels remains recommended only for RAG or starting/small maps, this module's token consumption far exceeds every other module combined and will easily consume all your context window if exported on the "Consolidate" or "Monolithic" modes.

~ REMOVED (deliberate simplification)

- "01. Project Audits" module and "Export Technical Debt (TODOs/FIXMEs)": technical debt reports, orphan/naming scans and _CRITICAL_WARNING injection consumed tokens re-stating what developers already know about their own project.
- "Copy AI Prompt" button, the Instructions_for_AI.md file and the embedded prompt: modern LLMs parse the payloads without a primer.
- "Export UI Visual Bindings" and the "Level of Detail (LOD)" selector: partial detail levels invited hallucinations; extraction now always runs at full structural depth (UMG bindings and widget trees are always included with the Blueprints module).
- Multi-language support (8 languages) was retired: the editor UI remains fully localizable (all texts use the localization system) but only English ships in V1.1.

~ TOKEN & METADATA OPTIMIZATION

Per-payload metadata was reduced from {"_METADATA":{"Tool":"...", "GeneratedAt":"..."}} to a minimal {"_Gen":"YYYY.MM.DD-HH.MM"} stamp. Combined with the always-on omission

of bulk numeric arrays, exports are significantly leaner.

~ ANALYTICS DASHBOARD CHANGES

- **New top tiles:** Total Time, Assets Exported, Files on Disk, Total Size (Disk).
Physical file count now comes from a real disk scan, so Monolithic/Consolidated exports show the true file count while the module chart still breaks down the individual assets (Assets-per-Module view).
- **New "Export Efficiency" panel:** Export Mode, Delta Serialization savings, Average size per asset (KB) and Physical files written.
- **New "Throughput & Output" panel:** Assets/sec throughput and estimated tokens.
- **Removed:** Peak RAM usage and all audit-only metrics (orphans, pending tasks, inheritance depth, tick-enabled actors).
- **Module breakdown tracks unique physical files** (the old per-write counter inflated counts on appended JSONL lines).

~ LOGGING OVERHAUL (FAB compliance)

- The "[ASSET CONTEXT EXPORTER FOR AI]" prefix spam was removed; the log category (LogContextExporter) already identifies the source.
- The raw process log (ContextExporter_Raw.log) is fixed: output chunks are now line-terminated instead of being concatenated into one endless line.
- The session manifest now records the engine version and the export mode.
- The background I/O writer thread uses an auto-reset event (the old manual-reset event caused a busy-poll loop after the first write).
- The exported folder tree is generated at the end of the LAST batch, so it now lists every file (it previously ran on the first batch only).

~ MULTI-VERSION ENGINE COMPATIBILITY (UE 5.4 - 5.7)

One single code base compiles across Unreal Engine 5.4, 5.5, 5.6 and 5.7.

The plugin detects the engine version at compile time and selects the matching APIs:

- World Partition iteration (ActorDescInstance API, 5.4+).

- SkeletalBodySetup header location (absent on 5.4, PhysicsEngine/ on 5.5+), resolved with `__has_include` guards.
- InstancedStruct location (StructUtils plugin module up to 5.4, CoreUObject on 5.5+), handled both in code and in the Build.cs module dependencies.
- Container shrink semantics (EAllowShrinking on 5.4+).

~ FIXES

- Fixed: an export could abort with return code 1 and produce nothing when the host project logged a benign "handled ensure" at engine startup (e.g. an AssetManager PrimaryAssetType whose class fails to load). The background worker now trusts the commandlet's explicit progress signals instead of the raw child exit code, and the scary startup callstack is collapsed into a single non-fatal notice line.
- Fixed: Physics Assets exported via naive reflection produced multi-megabyte payloads (constraint/body engine defaults, collision vertices). Replaced with a dedicated compact exporter (a full ragdoll is now a few KB).
- Fixed: AnimGraph node Details leaked editor-only pin-visibility metadata (ShowPinForProperties: bShowPin, PropertyFriendlyName, PropertyTooltip, ...) - now filtered out.
- Fixed: AnimBP non-animation function graphs were silently dropped from the export.
- Fixed: the `-NoMaterials` optimization flag was emitted but never parsed, so traced material/texture dependencies were processed even with visual modules disabled.
- Fixed: "Animations" module no longer artificially requires the Blueprints module.
- Fixed: FAB "NoAI" tag compliance is now enforced by default on fresh installs.
- Fixed: World Partition data is resolved through the proper engine accessor instead of a name-based object lookup.
- Fixed: dashboard "Data-Driven Rate" metric computed from unstable sorted indices (removed), and a number-formatting typo in the copied analytics summary.
- Fixed: in Fragmented mode a split file could be renamed to `_Part01` while its content was still sitting in the asynchronous write queue; renames now flush pending I/O first.
- Fixed: path aliasing only matched bare package names, so object references (`/Pkg/Asset.Asset` and `_C` class paths) were never compressed; both now resolve

through the same manifest token.

- Fixed: each batch process re-discovered the asset list from the registry, so batch indices and alias IDs could drift between processes; the counting phase now persists the exact list (deterministically sorted) and batches reuse it.
- Fixed: the content filter skipped any path containing the word "Localization" (silently omitting user folders such as /Game/LocalizationTest); it now only skips /L10N localized-content roots.
- Fixed: integer values of -1 / MAX were rewritten as "INDEX_NONE (No Limit)" even for legitimate gameplay values, and FName values lost their numeric suffixes; both now export verbatim.
- Fixed: Widget Blueprint properties were exported twice (per-node in the tree and in the flat widget list); the tree now carries topology only.
- Fixed: a failed commandlet launch left the UI waiting forever; it now reports the failure immediately. Editor asset compilation no longer freezes log/completion handling during an export.
- Fixed: settings persist to the per-user editor config layer instead of writing absolute user paths into the project's shared Default ini on every keystroke.
- Added: headless flags -DeepWorld=1, -MaxRAMMB=<n> and -NoAIFilter=0 so CI runs have full parity with the editor UI without depending on saved editor settings.

~ MULTI-VERSION NOTE (V1.1 hotfix)

User Defined Structs moved from the Engine module to CoreUObject/StructUtils in UE 5.5, handled by the same compile-time guard used for InstancedStruct.

The Extraction Engine & Context Optimization

The core process runs on an asynchronous background worker (a headless commandlet), allowing the Unreal Editor to remain responsive even during massive exports.

- **Smart Delta Serialization:** evaluates property values against their Class Default Object (CDO). Only variables explicitly modified by the developer are exported, reducing file size by up to 80%.
- **AST & Visual Logic Translation:** parses uncompiled visual programming, translating execution pins, pure math logic, and variable sets into strictly structured JSON.
- **Component Architecture Extraction:** reads the CDO and the Simple Construction Script to expose the exact hierarchy and spatial relationships of component templates.
- **Path Compression:** an aliasing system substitutes deep package paths (/Game/...) with compressed integer identifiers, saving tokens in the LLM's context window.

~Extraction Modules (V1.1)

- 01. Source Code (.cpp / .h):** raw C++ implementation and engine .ini configuration files (optionally including local plugin sources, subject to license validation).
- 02. Blueprints:** uncompiled logic, properties, execution links, component structures, Widget trees/bindings and the project dependency graph.
- 03. Data Assets:** Primary Data Assets, DataTables, CurveTables, StringTables, User Defined Structs, User Defined Enums and the project Gameplay Tag dictionary.
- 04. World Levels:** actor transforms, localized properties, and tags within .umap environments (with optional Deep World Partition load), plus Static Mesh metadata (LODs, bounds, material slots, sockets, collision).
- 05. VFX & Materials:** Niagara systems, material graphs (math AST) and material instances.
- 06. Animations:** Skeletons, Skeletal Meshes, Physics Assets, Anim Blueprints (state machines included), Blend Spaces, Sequences, Montages, Level Sequences, Control Rigs and IK Rigs/Retargeters.
- 07. AI Logic:** Behavior Trees, Blackboards, EQS and StateTrees.
- 08. Audio:** MetaSound graphs and SoundCue structures.
- 09. Input Mappings:** Enhanced Input contexts and actions.
- 10. Physics:** Physical Materials.

Quick Start Guide (V1.1)

1. Launch your Unreal Engine project.
2. On the top menu bar, navigate to Tools > Asset Context Exporter For AI.
3. Select a strategic preset (e.g., Core Architecture or Full Project) or toggle modules.
4. Under Export Settings, pick ONE Export Mode: Standard, Fragmented, Consolidated or

Monolithic.

5. Define the output directory and click RUN CONTEXT EXPORT.

6. Once finished, use Open Analytics Dashboard to review the export, or Open Export Folder to attach the payloads to your AI model of choice.

Marketplace Compliance

The plugin incorporates an active safety mechanism (bExcludeNoAIAssets, enabled by default) designed to respect Epic Games FAB "NoAI" directives, automatically bypassing flagged assets during extraction. Disabling this protection transfers all technical and legal responsibility to the user's environment. Activating "Include Plugins" requires an explicit in-editor confirmation acknowledging third-party EULA obligations.

Technical Specifications (V1.1)

- **Supported Platforms:** Windows 64-bit (Editor-only plugin)
- **Supported Engine Versions:** 5.4, 5.5, 5.6, 5.7 (single code base with compile-time version detection)
- **Output:** UTF-8 JSONL / Markdown, always English folder & file names
- **Languages:** English only (localized UI may return in a future release)

Contact

mhvan_mx@outlook.com

DOCUMENTATION V1.0 (LEGACY - superseded by V1.1)

Overview & Business Value AI Context Exporter is a comprehensive C++ solution designed for solo developers, indie creators, and AAA studios looking to standardize their Data-Driven Pipeline. Built with the philosophy that leveraging advanced AI shouldn't require a computer science degree, the tool translates Unreal Engine binary files (.uasset, .umap) into highly optimized, machine readable text formats (JSONL). It serves as the ultimate bridge between project architecture and Large Language Models (LLMs) or Agentic AI, empowering both non-programmers and seasoned engineers to build, audit, and scale their projects efficiently.

Enterprise-Grade Security & Air-Gapped Operation: The tool operates 100% offline within the studio's intranet. There are no external dependencies, third-party APIs, or hidden telemetry. Proprietary code and project logic never leave the local hardware, ensuring strict compliance with corporate Non-Disclosure Agreements (NDAs).

Objective Code Audits & Architectural Health: Beyond AI extraction, the engine functions as a technical debt scanner. It

enables Technical Directors to perform Code Reviews on visual Blueprints using quantifiable metrics, saving dozens of hours in debugging massive projects. [REMOVED IN V1.1]

API Cost Optimization (RAG Pipeline): Generating AI context can become cost-prohibitive if "noise" or redundant engine data is exported. Through advanced compression and delta serialization, this tool massively reduces the payload size (JSONL), directly decreasing token consumption and billing costs on platforms like OpenAI, Anthropic, or Vector Database systems.

Zero Friction: No Python environments to configure and no external dependencies. A drop-in installation that produces .jsonl files ready to be integrated into any workflow.

Architectural Health & Project Audits [REMOVED IN V1.1]: Blueprint Node Complexity scoring, Technical Debt & Orphan Assets reports, Performance & Hierarchy warnings (Data-Driven Rate, Max Inheritance Depth, Tick-enabled Actors).

The Extraction Engine & Context Optimization: asynchronous background Worker, Smart Delta Serialization (CDO diff), AST & Visual Logic Translation, Component Architecture Extraction (ECS), Path Compression.

Integrated Multilingual Support [REMOVED IN V1.1]: English (en), Japanese (ja), Simplified Chinese (zh-Hans), Korean (ko), German (de), Spanish (es), French (fr), Brazilian Portuguese (pt-BR). Output folder structure invariably generated in English.

Extraction Modules (V1.0): Source Code (.cpp/.h), Local Plugins, Blueprints (Logic & Nodes), World Levels (Actors & Partitions), VFX & Materials, Animations (Animation Blueprints, Sequences, Montages), AI Logic (BT/Blackboards/StateTrees), Audio Sounds (MetaSounds/SoundCues), Data Assets (Primary Data Assets, Data Tables, String Tables), Input Mappings (Enhanced Input), Physics (Physical Materials).

Interactive Analytics Dashboard: Module Breakdown (disk size and file count by asset type), Top 20 Rankings (most complex Blueprints, heaviest memory assets, most referenced assets).

Quick Start Guide (V1.0): Tools > Asset Context Exporter For AI; select preset; define format (Monolithic JSONL for vector databases or Fragmented for standard context windows); define output directory; RUN CONTEXT EXPORT; Open Export Folder.

Marketplace Compliance: active safety mechanism (bExcludeNoAIAssets) respecting Epic Games FAB "NoAI" directives.

Technical Specifications (V1.0):

Windows 64-bit;

Supported Engine Versions: 5.5, 5.6, 5.7.

Contact

mhvan_mx@outlook.com

